

可以。这个演化路线其实非常有意思，因为它代表了 **RLHF / LLM 后训练方法** 的一次重大变化：

$$\boxed{\text{PPO} \rightarrow \text{DPO} \rightarrow \text{GRPO}}$$

但这里要先提醒一个容易混淆的地方：

DPO 并不是 PPO 的“下一代版本”，GRPO 也不是简单的 DPO 改进。

它们解决的问题不同：

- **PPO**：显式做 RL，需要 reward model + value model
- **DPO**：把 RL 优化问题转化成一个直接的 preference classification / likelihood optimization 问题，不需要在线 RL
- **GRPO**：重新回到 policy optimization，但针对 LLM 推理任务，用同一个问题采样多个答案形成 **group**，用 group-relative reward 代替 value model

所以更准确的路线是：

$$\boxed{\text{PPO} \longrightarrow \text{DPO}}$$

以及：

$$\boxed{\text{PPO} \longrightarrow \text{GRPO}}$$

而不是严格意义上的：

$$PPO \rightarrow DPO \rightarrow GRPO$$

不过从 LLM post-training 的历史和思想演化来看，把它们放在一起理解非常合适。

一、先回顾 PPO：LLM 为什么能用 PPO？

我们先把上一节的 PPO 映射到 LLM。

传统 RL：

$$s_t \rightarrow a_t \rightarrow r_t$$

到了 LLM：

$$\boxed{s_t = \text{prompt} + \text{已生成的 tokens}}$$

而：

$$\boxed{a_t = \text{下一个 token}}$$

所以一个完整回答：

$$y = (y_1, y_2, \dots, y_T)$$

实际上就是一条 trajectory。

例如：

Prompt:

“解释一下为什么天空是蓝色的”

↓

Token 1

↓

Token 2

↓

Token 3

↓

...

↓

EOS

因此 LLM 可以被看成一个 RL agent。

二、PPO 在 LLM 里到底优化什么？

假设：

$$\pi_{\theta}(y|x)$$

是当前语言模型。

我们希望：

回答质量越来越高

所以有一个 Reward Model：

$$r_{\phi}(x,y)$$

给回答评分。

例如：

Prompt

— Answer A → RM → 8.7

— Answer B → RM → 3.2

于是 PPO 希望：

$$\pi_{\theta}$$

提高高 reward answer 的概率。

三、但 LLM 的 PPO 有一个非常重要的问题

如果只优化：

$$r_{\phi}(x, y)$$

模型可能会：

钻 Reward Model 的漏洞。

例如 Reward Model 喜欢：

“非常详细、非常有礼貌、结构非常完整”

那么模型可能疯狂输出：

“非常非常非常非常详细.....”

即使答案本身并没有变好。

所以 PPO 通常还要限制：

$$\pi_{\theta}$$

不要偏离原始 SFT model 太远。

这就是 KL penalty。

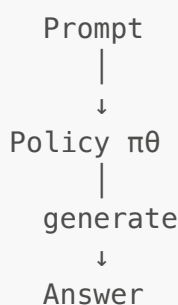
典型目标：

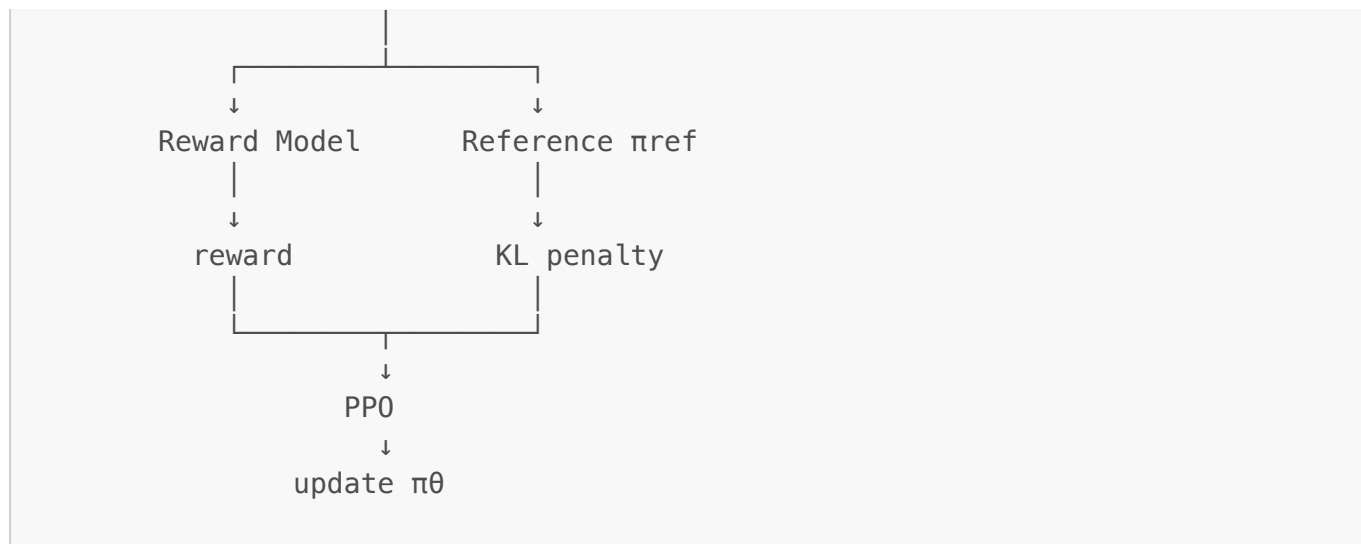
$$\max_{\theta} \mathbb{E}[r_{\phi}(x, y) - \beta D_{KL}(\pi_{\theta} || \pi_{\text{ref}})]$$

其中：

- π_{θ} ：正在训练的 policy
- π_{ref} ：reference model，通常是 SFT model
- r_{ϕ} ：reward model
- β ：KL 惩罚系数

所以 PPO-RLHF 大致是：





四、PPO 的巨大问题：它太复杂了

你现在应该能看到 PPO-RLHF 有多少组件：

SFT model + Policy + Reward Model + Value Model + Rollout + PPO + KL

这意味着训练成本非常高。

尤其是 LLM：

- 模型巨大
- rollout 贵
- reward model 要训练
- value model 要训练
- PPO 本身比较复杂
- online sampling 成本高

于是自然出现一个问题：

我们真的需要把 preference optimization 写成 RL 吗？

DPO 的回答：

不一定。

五、DPO：Direct Preference Optimization

DPO：

Direct Preference Optimization

它的核心思想非常漂亮：

我们有 chosen / rejected preference 数据，为什么不直接训练模型让 chosen 的概率高于 rejected?

假设数据：

$$(x, y_w, y_l)$$

其中：

- x : prompt
- y_w : winner / chosen answer
- y_l : loser / rejected answer

例如：

Prompt:
“解释量子纠缠”

Chosen:
“量子纠缠是.....”
↑
人类更喜欢

Rejected:
“量子纠缠就是两个粒子.....”
↑
人类不喜欢

DPO 想做：

$$\pi_{\theta}(y_w|x) > \pi_{\theta}(y_l|x)$$

六、最直觉的 DPO Loss

最简单的理解是：

$$\log \pi_{\theta}(y_w|x) - \log \pi_{\theta}(y_l|x)$$

我们希望：

$$\log \pi_{\theta}(y_w|x) - \log \pi_{\theta}(y_l|x)$$

越来越大。

但是 DPO 不仅看当前 policy。

它还引入 reference model:

$$\pi_{\text{ref}}$$

于是比较：

$$\frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)}$$

和：

$$\frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)}$$

七、DPO 最核心的公式

DPO loss：

$$L_{\text{DPO}} = -\mathbb{E} \left[\log \sigma \left(\beta \left[\log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right] \right) \right]$$

这个公式第一次看很吓人。

我们拆开。

定义：

$$\Delta_{\theta} = \log \pi_{\theta}(y_w|x) - \log \pi_{\theta}(y_l|x)$$

以及：

$$\Delta_{\text{ref}} = \log \pi_{\text{ref}}(y_w|x) - \log \pi_{\text{ref}}(y_l|x)$$

那么 DPO 就是：

$$L_{\text{DPO}} = -\log \sigma [\beta(\Delta_{\theta} - \Delta_{\text{ref}})]$$

这就简单很多。

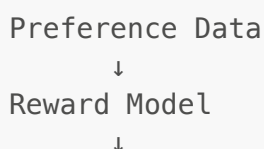
它实际上在要求：

Policy 相对于 Reference，更偏爱 chosen

八、为什么 DPO 不需要 Reward Model?

这是 DPO 最关键的方。

PPO：



Reward
↓
PPO
↓
Policy

DPO:

Preference Data
↓
Chosen / Rejected
↓
DPO Loss
↓
Policy

所以 DPO 不需要单独训练:

$$r_\phi$$

因此也不需要:

Reward Model

同时也不需要:

Value Model

也不需要:

PPO rollout + advantage estimation

这使得训练大幅简单化。

九、DPO 为什么“看起来不像 RL”，却能解决 RLHF 问题？

这是 DPO 最漂亮的数学思想。

传统 RLHF:

$$\max_{\pi} \mathbb{E}_{y \sim \pi} [r(x, y)] - \beta D_{KL}(\pi(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x))$$

对于固定 reward, 最优策略存在一个解析形式:

$$\pi^*(y|x) \propto \pi_{\text{ref}}(y|x) \exp\left(\frac{r(x, y)}{\beta}\right)$$

取 $\log$:

$$\log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} = \frac{r(x, y)}{\beta} + C(x)$$

因此:

$$r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \text{constant}$$

也就是说:

$$\boxed{Reward \leftrightarrow \log \frac{\pi}{\pi_{\text{ref}}}}$$

这就是 DPO 的核心数学桥梁。

十、Preference Model 又是什么?

假设 Bradley-Terry preference model:

$$P(y_w \succ y_l | x) = \sigma(r(x, y_w) - r(x, y_l))$$

把刚才的:

$$r(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$$

代进去:

$$P(y_w \succ y_l | x) = \sigma \left(\beta \left[\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right] \right)$$

然后最大化 chosen 被偏好的概率。

这就直接得到 DPO。

所以:

$$\boxed{DPO = \text{把 KL-regularized RL 的最优策略条件} + \text{Preference Model} \rightarrow \text{直接优化 Policy}}$$

十一、到这里 PPO $\rightarrow$ DPO 的核心变化

可以这样理解:

PPO:

问:

“这个回答 reward 有多少? 我要通过 RL 把 reward 最大化。”

DPO:

问:

“人类更喜欢 A 还是 B? 我直接训练模型让 A 的相对概率超过 B。”

所以:

PPO : Reward-driven

而:

DPO : Preference-driven

十二、但是 DPO 有一个很明显的问题

DPO 特别依赖:

offline preference data

例如:

```
Prompt
├── chosen
└── rejected
```

这些数据通常来自:

- 人类标注
- GPT judge
- 其他 reward / preference pipeline

问题是:

如果我要训练模型做数学题、代码题、推理题，为什么一定要人工告诉我哪个答案更好？

因为很多任务可以:

自动验证

比如数学题:

```
问题:
 $x^2 - 5x + 6 = 0$ 

Answer A:
x=2,3
→ correct
```

Answer B:
x=1,6
→ wrong

我们根本不需要人类 preference。

可以直接：

$$\text{reward}(A) = 1$$

$$\text{reward}(B) = 0$$

于是又回到了 RL。

十三、这就是 GRPO 出现的背景

GRPO：

Group Relative Policy Optimization

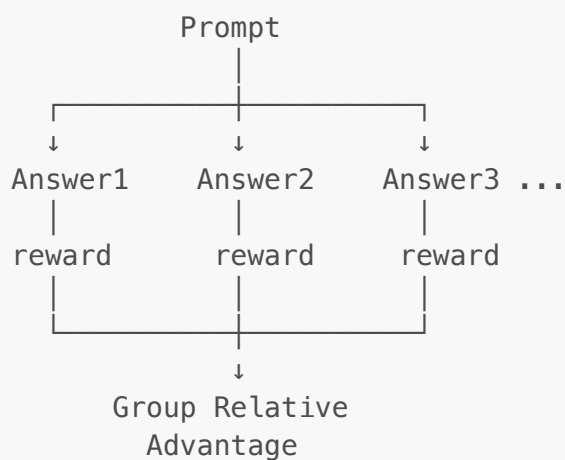
它最重要的思想：

同一个 prompt，不只生成一个答案，而是生成一组答案，然后比较这一组答案的 reward。

例如：

Prompt:

“解方程 $x^2 - 5x + 6 = 0$ ”



例如：

$$r_1 = 1$$

$$r_2 = 0$$

$$r_3 = 1$$

$$r_4 = 0$$

计算 group mean:

$$\bar{r} = \frac{1}{4}(1 + 0 + 1 + 0) = 0.5$$

然后:

$$A_i \approx r_i - \bar{r}$$

所以:

$$A_1 = 0.5$$

$$A_2 = -0.5$$

$$A_3 = 0.5$$

$$A_4 = -0.5$$

于是:

比 group 平均水平好的回答 ↑ 比 group 平均水平差的答案 ↓

十四、GRPO 最关键的创新: 不要 Value Model

这是 GRPO 和 PPO 最大的区别之一。

PPO:

$$A_t \approx Q(s_t, a_t) - V(s_t)$$

所以需要:

$$V_\phi(s)$$

来估计 baseline。

而 GRPO:

我直接拿同一个 prompt 的多个回答进行比较。

所以:

Group itself acts as a baseline

例如:

$$r_1, r_2, \dots, r_G$$

计算:

$$\mu = \frac{1}{G} \sum_{i=1}^G r_i$$

然后：

$$A_i = \frac{r_i - \mu}{\sigma + \epsilon}$$

这就是 group-relative advantage 的典型形式。

因此：

不需要单独训练 Value Model

十五、这解决了 LLM RL 的一个巨大问题

PPO：

```
Policy Model
+
Value Model
+
Reward Model
+
Reference Model
```

GRPO：

```
Policy Model
+
Reference Model
+
Reward Function
```

少了：

Value Model

这对于大语言模型非常重要。

因为如果 policy 是几十/几百/上千亿参数：

再维护一个同规模 value model，显存和训练成本都很高。

GRPO 的 group relative idea 可以绕开它。

十六、GRPO 的 Policy Objective

它仍然保留 PPO 的核心思想：

$$r_i(\theta) = \frac{\pi_\theta(y_i|x)}{\pi_{\text{old}}(y_i|x)}$$

然后：

$L^{GRPO} \sim \min(r_i A_i, clip(r_i, 1 - \epsilon, 1 + \epsilon) A_i)$

所以你会发现：

$GRPO \text{ 的核心 policy update 非常像 PPO}$

真正变化的是：

$A_i \text{ 怎么得到}$

十七、PPO vs GRPO：最关键区别

PPO：

$$A_t = Q(s_t, a_t) - V(s_t)$$

通常：

$$V(s_t)$$

来自 Value Network。

GRPO：

$A_i = \frac{r_i - \text{mean}(r)}{\text{std}(r) + \epsilon}$

其中 i 是同一个 prompt 下的一条 response。

所以：

	PPO	GRPO
Policy	✓	✓
Reference	常见于 RLHF	✓
Reward	✓	✓
Value Model	✓	✗
Advantage	Critic/GAE	Group relative

	PPO	GRPO
Multiple samples per prompt	不要求	核心
Clipping	✓	✓
KL regularization	常见	常见

十八、为什么 Group Relative 有意义？

假设一个问题生成四个回答：

$$r = [0.9, 0.8, 0.2, 0.1]$$

那么：

$$\bar{r} = 0.5$$

得到：

$$A = [0.4, 0.3, -0.3, -0.4]$$

所以 GRPO 不只是问：

“这个答案绝对好不好？”

而是问：

“在这次生成的候选答案里面，它相对其他答案好不好？”

这对于 reasoning 特别有用。

十九、为什么 GRPO 特别适合数学 / Code / Reasoning？

因为这些任务存在：

Verifiable Reward

例如数学：

answer correct?

代码：

pass unit tests?

棋类：

win/loss

定理证明：

proof valid?

这些任务可以自动产生 reward。

于是：

```
Prompt
↓
Policy
↓
Generate G answers
↓
Verifier
↓
r1 r2 ... rG
↓
Group normalization
↓
Advantages
↓
GRPO
↓
Policy update
```

完全不需要：

人工 preference

这和 DPO 非常不同。

二十、现在把 DPO 和 GRPO 放在一起

你可以发现：

DPO

输入：

(x, y_w, y_l)

也就是：

pairwise preference

告诉模型：

A 比 B 好。

GRPO

输入：

$$(x, y_1, \dots, y_G)$$

以及：

$$(r_1, \dots, r_G)$$

告诉模型：

在这一组答案里面，哪些相对更好。

所以：

DPO : pairwise preference

GRPO : group-wise reward comparison

二十一、一个非常重要的统一视角

现在我们把三个算法放到同一个框架里：

PPO

目标：

maximize expected reward

需要：

RewardModel

并且：

ValueModel

帮助估计：

A

DPO

目标：

maximize preference likelihood

不需要显式 reward model。

直接使用：

$$(y_w, y_l)$$

GRPO

目标：

maximize verifiable/group-relative reward

不需要 Value Model。

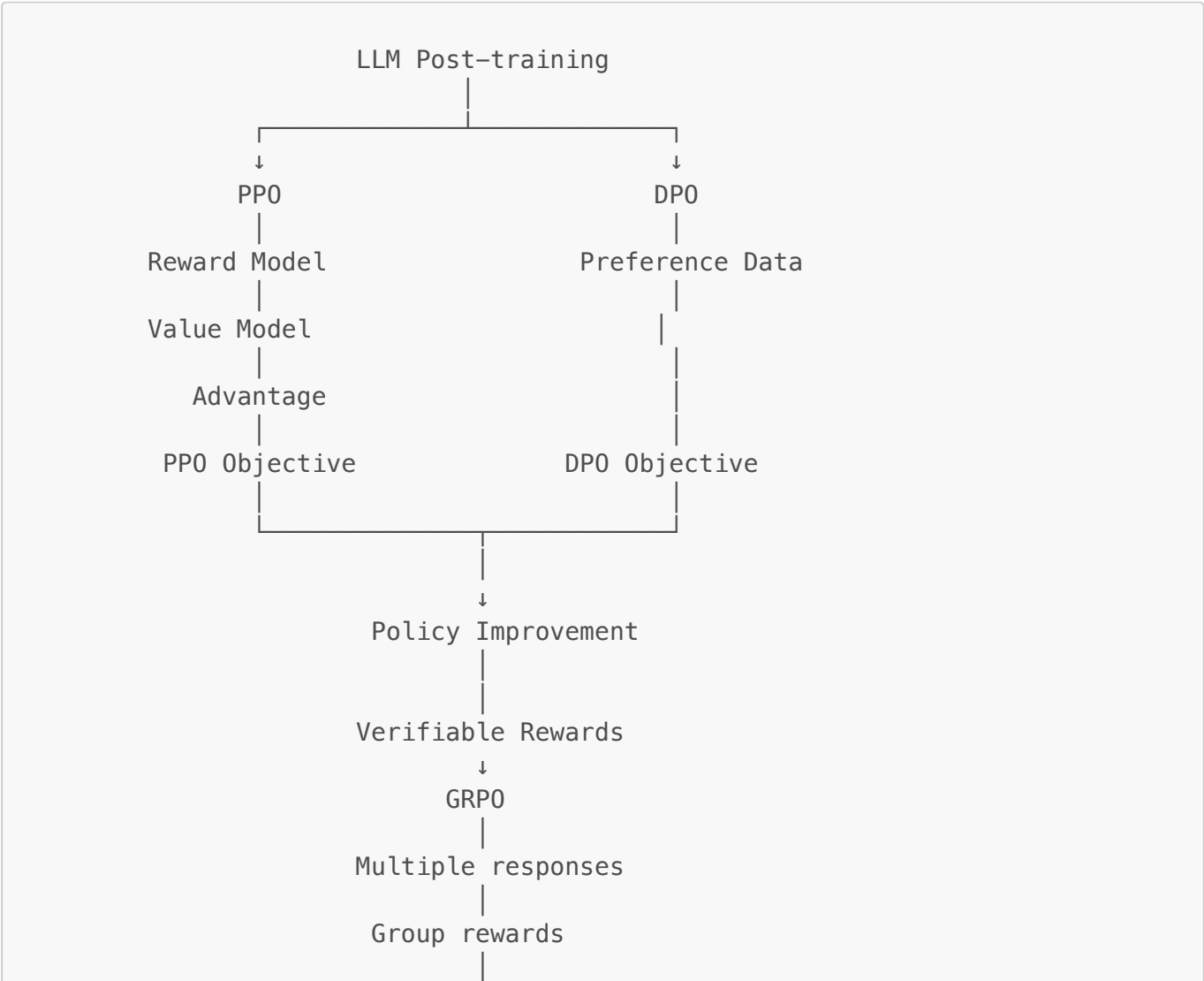
通过：

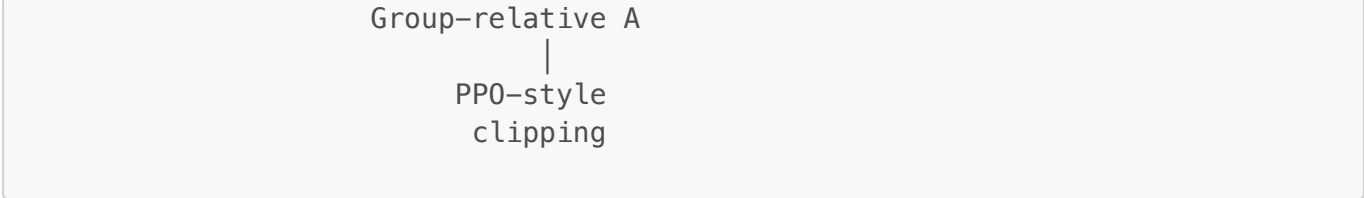
$$\{y_1, \cdots, y_G\}$$

构造：

$$A_i$$

二十二、三个算法的结构图





二十三、最值得你理解的是：三种“评价方式”

实际上 PPO、DPO、GRPO 最大区别可以浓缩成：

怎么告诉模型“这个回答好不好”？

PPO

Reward Model 告诉它：

$r(x, y) = 8.3$

绝对评分。

DPO

Preference 数据告诉它：

$y_w \succ y_l$

相对偏好。

GRPO

Verifier / Reward 告诉它：

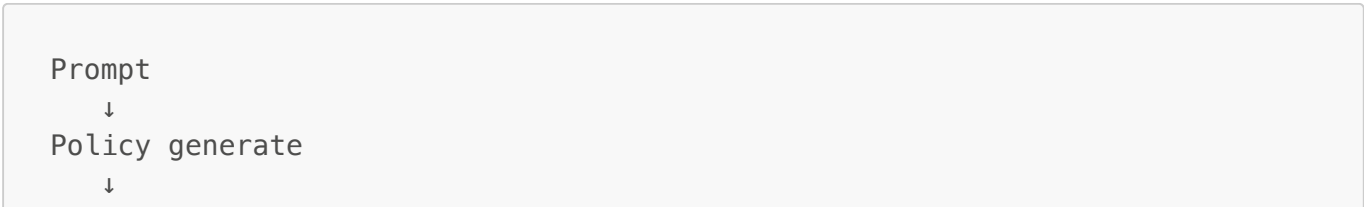
$r_1, r_2, \cdots, r_G$

然后在 group 内进行相对比较。

二十四、从训练数据角度看

这个区别更加直观。

PPO



```
Answer
  ↓
Reward Model
  ↓
reward
  ↓
PP0
```

所以是：

Online RL

DPO

```
Prompt
  ↓
Offline dataset
  ↓
Chosen / Rejected
  ↓
DP0
```

所以：

Offline preference optimization

GRPO

```
Prompt
  ↓
Policy generate G answers
  ↓
Verifier
  ↓
G rewards
  ↓
Group advantage
  ↓
GRPO
```

所以：

Online RL + group-relative baseline

二十五、为什么 DPO 不像 GRPO 那样在线生成？

因为 DPO 的训练目标是：

$$(x, y_w, y_l)$$

它需要的是：

已经知道哪个更好。

而 GRPO：

$$x \rightarrow \{y_1, y_2, \dots, y_G\}$$

然后：

$$\text{reward}(y_i)$$

所以 GRPO 本身就在训练过程中不断探索：

当前 policy $\rightarrow$ 新答案 $\rightarrow$ reward $\rightarrow$ 更新 policy

这就是 RL 的味道。

二十六、从“探索”角度理解三者

这是另一个非常重要的区别。

DPO

主要学习：

数据里已经存在的 preference。

如果 dataset 里没有一种行为：

模型很难自己发现它。

PPO

可以：

Policy $\rightarrow$ 探索 $\rightarrow$ Reward

所以能发现新的高 reward trajectory。

GRPO

更加明确地：

一个 prompt $\rightarrow$ 多个 reasoning trajectories

然后比较。

因此对于 reasoning:

探索能力非常重要

二十七、一个数学题例子把三者彻底串起来

问题:

$$x^2 - 5x + 6 = 0$$

答案:

Answer A

$$x = 2, 3$$

Answer B

$$x = 1, 6$$

Answer C

$$x = 2, 3$$

DPO

人类标注:

$$A \succ B$$

于是:

$$L_{DPO}$$

推动:

$$\pi(A|x) \uparrow$$

$$\pi(B|x) \downarrow$$

PPO

Reward Model:

$$r(A) = 1$$

$$r(B) = 0$$

然后：

$$A_t$$

通过 Critic / GAE 估计。

最后 PPO 更新。

GRPO

一次生成：

$$A, B, C, D$$

得到：

$$r = [1, 0, 1, 0]$$

mean：

$$\bar{r} = 0.5$$

所以：

$$A = [0.5, -0.5, 0.5, -0.5]$$

然后：

$$A > 0$$

的 reasoning trajectory 概率增加。

$$A < 0$$

的 reasoning trajectory 概率降低。

二十八、所以为什么 DeepSeek 类 reasoning training 会喜欢 GRPO？

核心原因可以简单理解成：

$$\boxed{\textit{Reasoning} + \textit{VerifiableReward} + \textit{LargeLLM}}$$

这个组合非常适合 GRPO。

因为：

1. Reward 容易自动获得

数学：

correct/incorrect

代码：

tests pass/fail

2. 不需要 Value Model

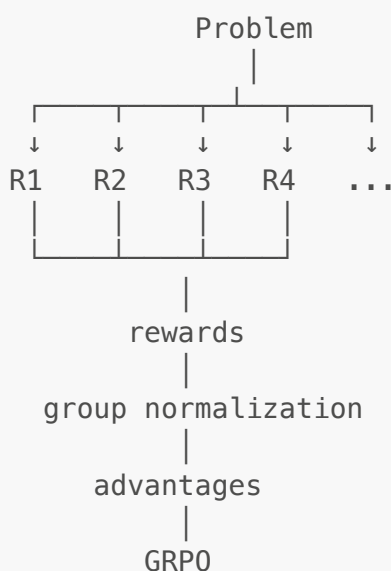
节省巨大模型资源。

3. 可以一次产生多个 reasoning trajectories

例如：

$$G = 8$$

一个问题：



这天然鼓励：

探索不同 reasoning path。

二十九、但 GRPO 并不是“完美 PPO”

GRPO 也有自己的问题。

因为它没有真正的：

$$V(s)$$

所以 group-relative advantage：

$$A_i = \frac{r_i - \mu}{\sigma + \epsilon}$$

本质上是一个：

$$\boxed{relativebaseline}$$

而不是精确的：

$$Q(s, a) - V(s)$$

因此：

- group size 很重要
- reward 分布很重要
- 如果一组答案全部错，可能没有好的正向 signal
- 如果一组答案全部对，也可能缺乏区分度

例如：

$$r = [0, 0, 0, 0]$$

那么：

$$A_i \approx 0$$

这一组几乎没有 policy learning signal。

反过来：

$$r = [1, 1, 1, 1]$$

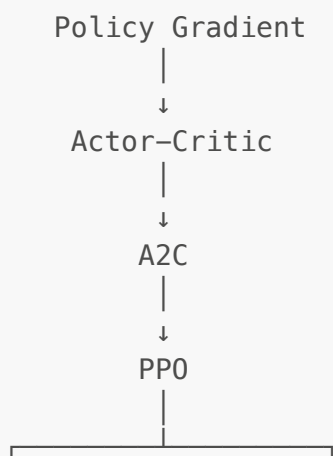
也一样。

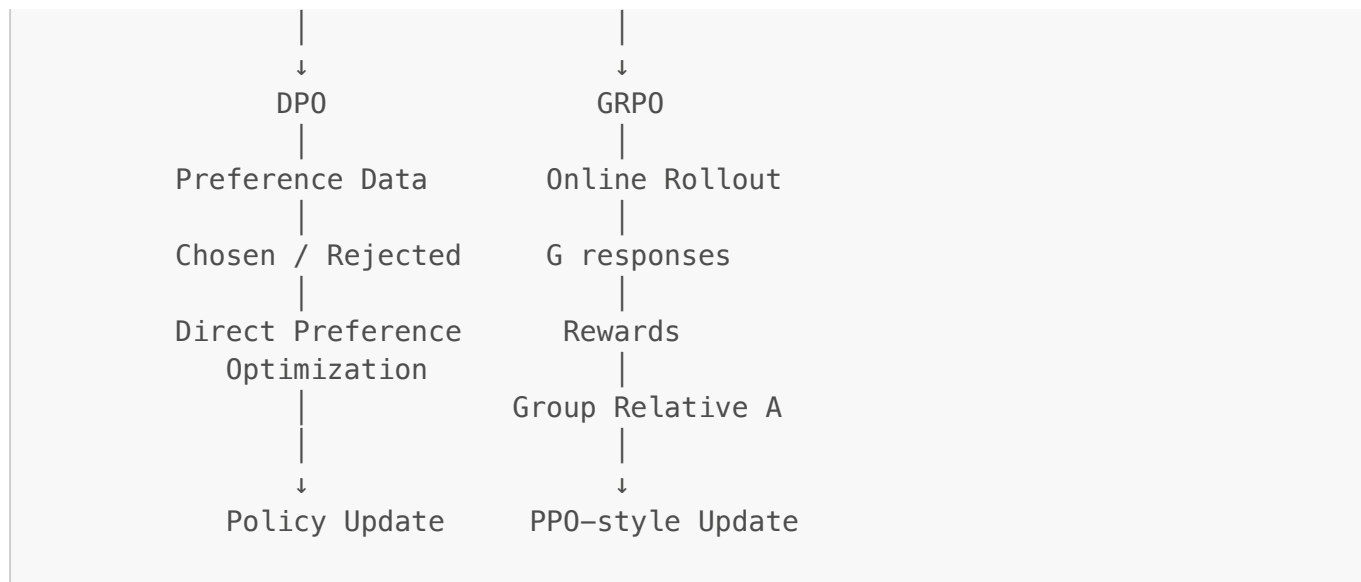
所以 GRPO 的关键之一就是：

$$\boxed{group内必须存在一定的rewarddiversity}$$

三十、最后给你一个“从 PG 到 GRPO”的完整知识树

你现在可以把整个 RL / LLM post-training 体系理解成：





三十一、把它们压缩成一句话

PPO

“Reward Model 说这个回答值多少分，我用 RL 最大化这个分数。”

DPO

“人类说 A 比 B 好，我直接让模型更偏向 A。”

GRPO

“我让模型自己生成一组答案，用 verifier/reward 比较它们，再强化相对更好的答案。”

三十二、最核心的演化逻辑

真正值得记住的是：

PPO

解决：

如何稳定地进行 RL policy optimization?

↓

DPO

发现：

如果我们已经有 preference pair，可以绕过复杂的 RL pipeline，直接优化 policy。

↓

GRPO

又发现：

对于数学、代码、推理这种有可验证 reward 的任务，我们不一定需要人工 preference，也不一定需要昂贵的 Value Model；让 policy 一次生成多个答案，然后做 group-relative advantage 就可以进行 RL。

因此三者实际上对应三种不同的监督信号：

PPO : absolute reward

DPO : pairwise preference

GRPO : group-relative reward

而它们最终的目的完全一致：

让 π_θ 越来越倾向于产生高质量回答。

如果你前面问的 **Policy Gradient** → **A2C** → **PPO** 和这里的 **PPO** → **DPO / GRPO** 再往下连，下一层最值得搞懂的是“为什么 **DPO** 的公式可以从 **KL-regularized RL** 推导出来，以及 **GRPO** 的 **group advantage** 为什么可以替代 **PPO** 的 **value function**”。这两个推导是理解 **DPO** 和 **GRPO** 不是背公式，而是真正理解其本质的关键。